



Jin L.C. Guo

Content Review

by Deeksha Arya

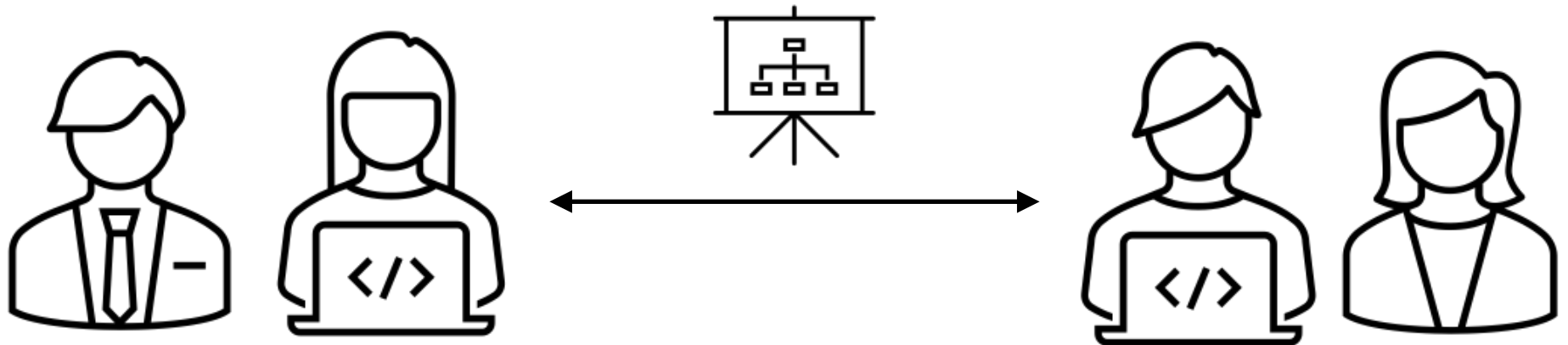
Image Source: https://upload.wikimedia.org/wikipedia/commons/2/2b/Cepaea_nemoralis_active_pair_on_tree_trunk.jpg

What we'll (try to) cover today:

- UML Diagrams
 - Why are they important?
 - Structure and Conventions
- Interfaces
 - Recognizing when to use an interface
- Dealing with duplicates or unique objects
 - “Equals” vs. Flyweight
 - What IS the hashCode method?
- Unit Testing
- Clarifying Iterable vs Iterator
- Clarifying Comparable vs Comparator

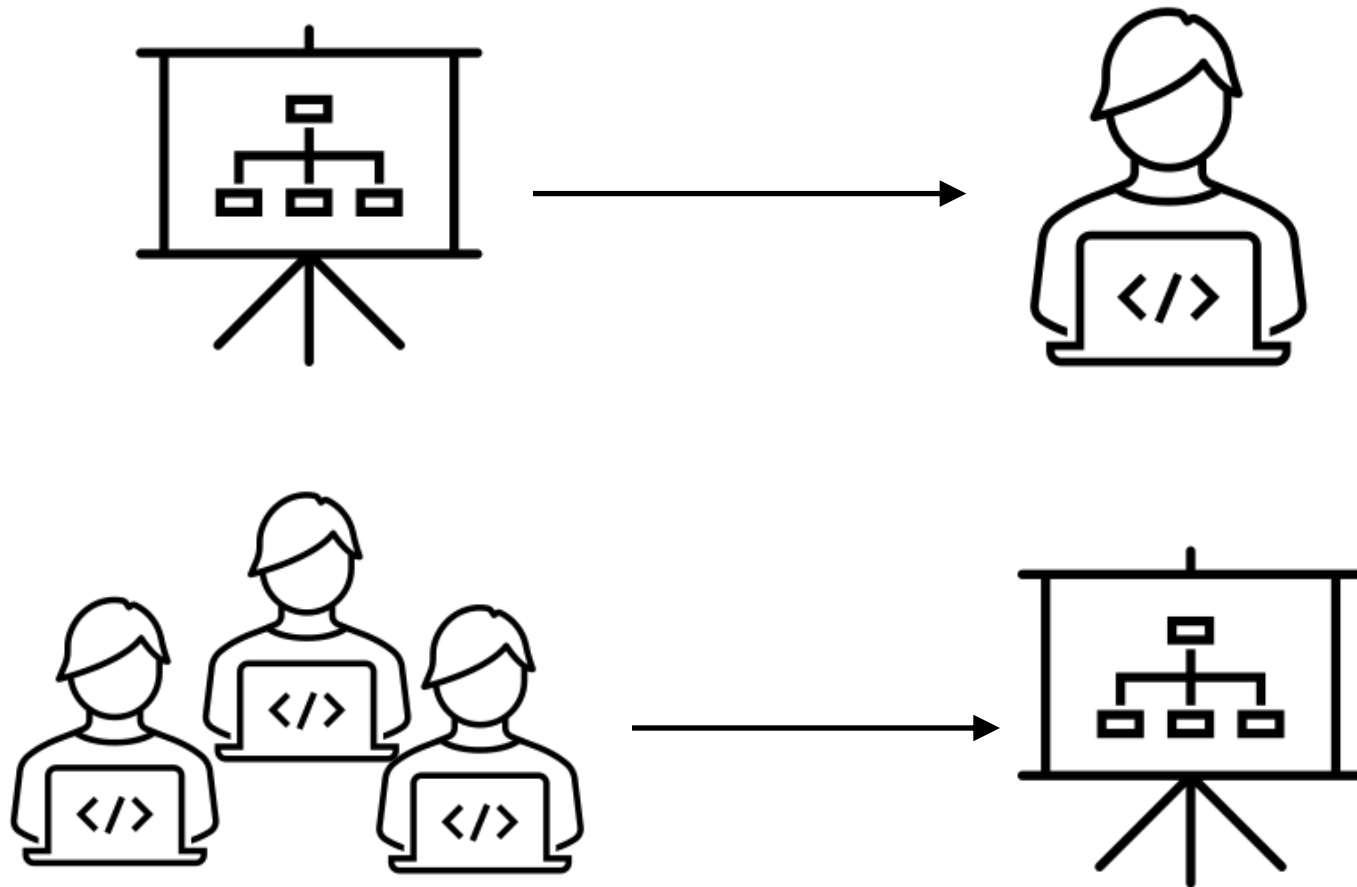
UML Diagrams

Why are they important?

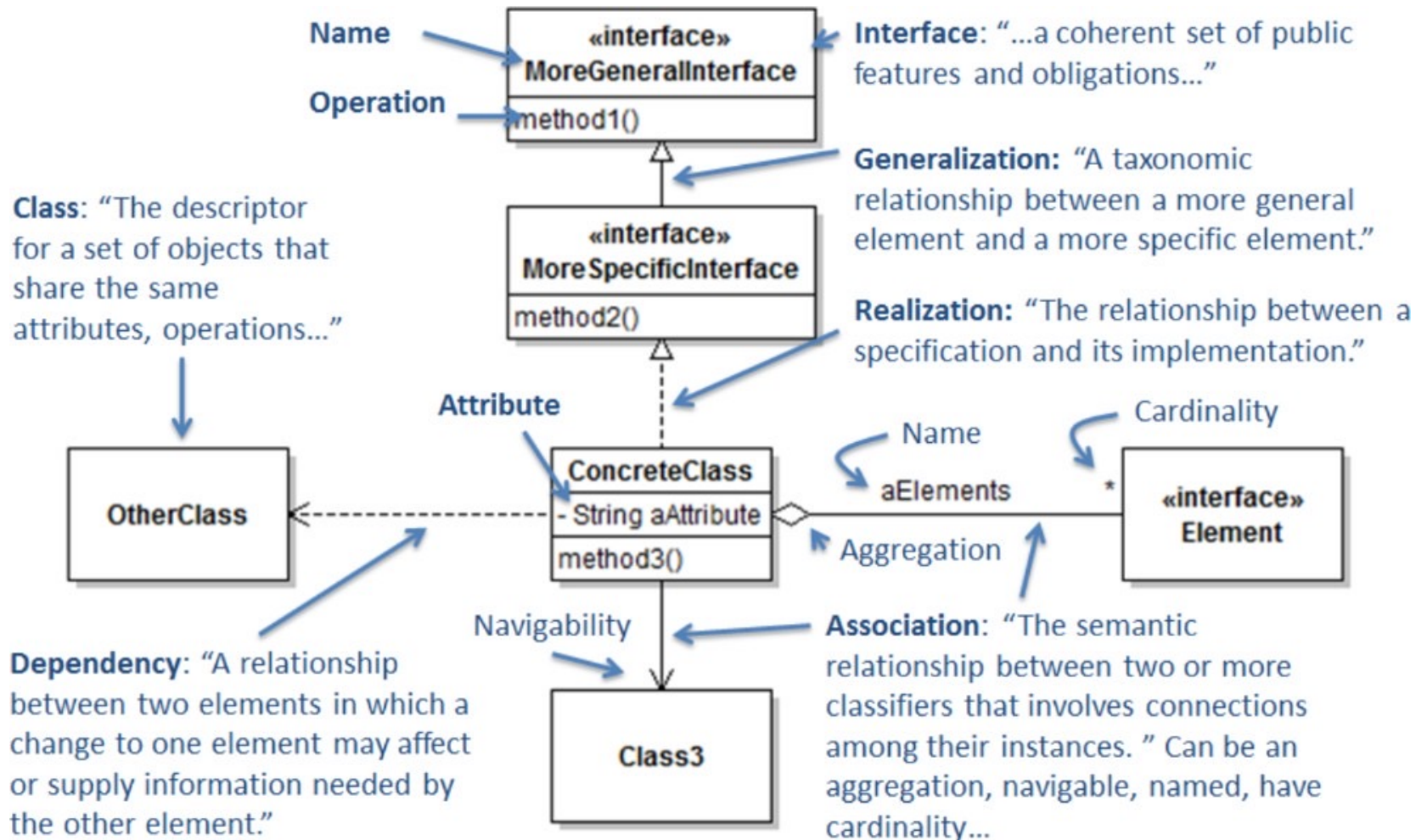


UML Diagrams

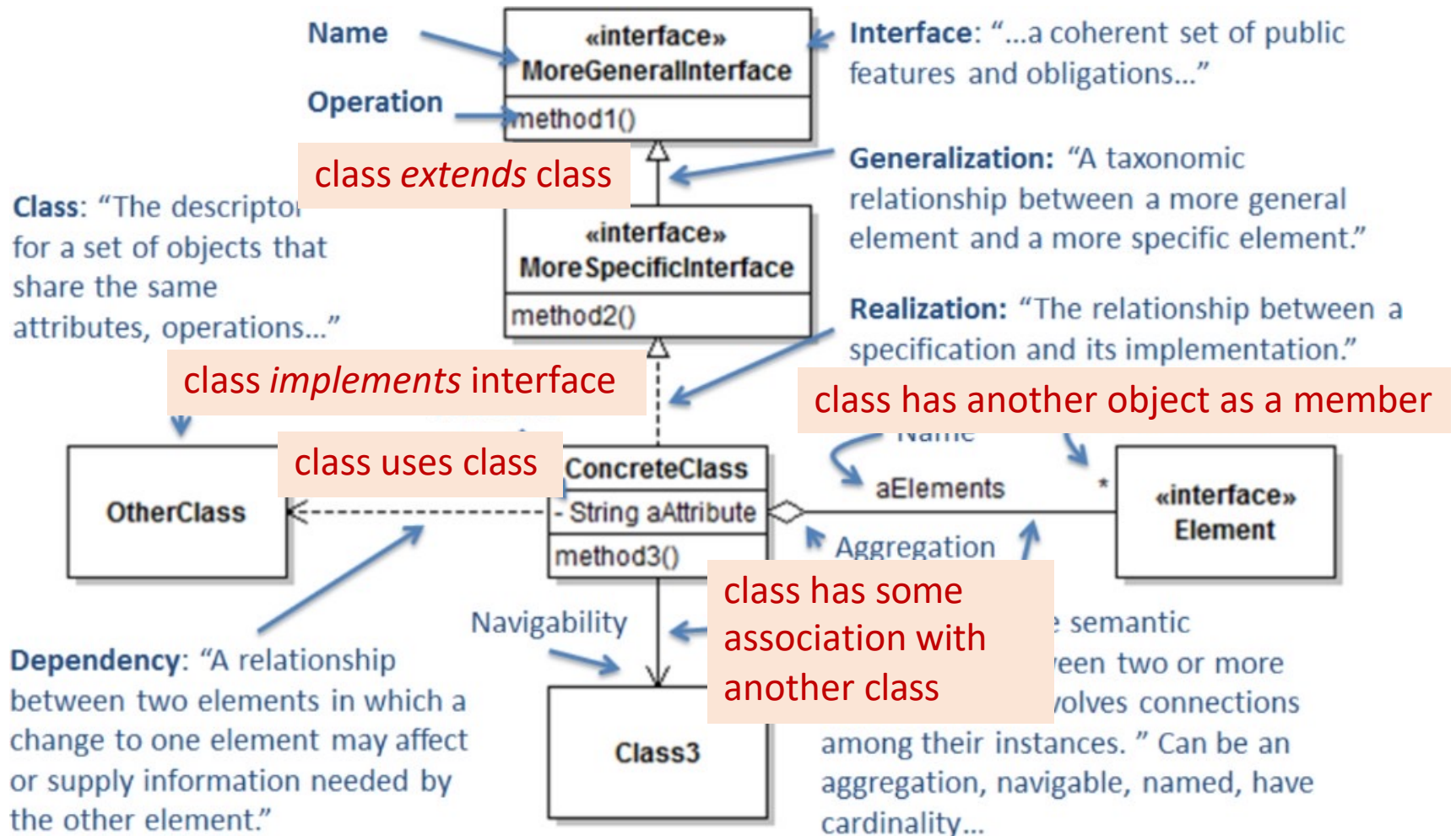
Why are they important?

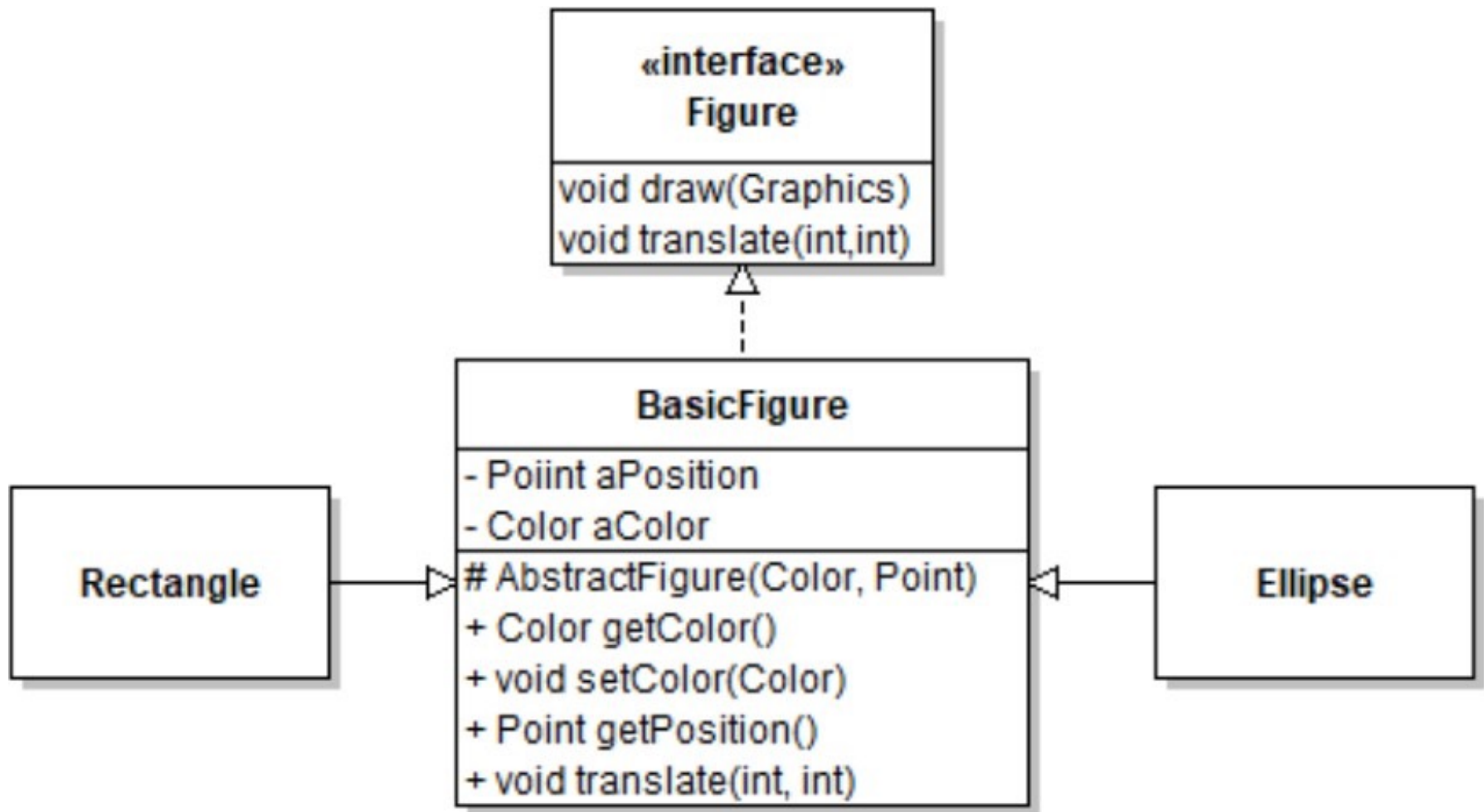


Class Diagram



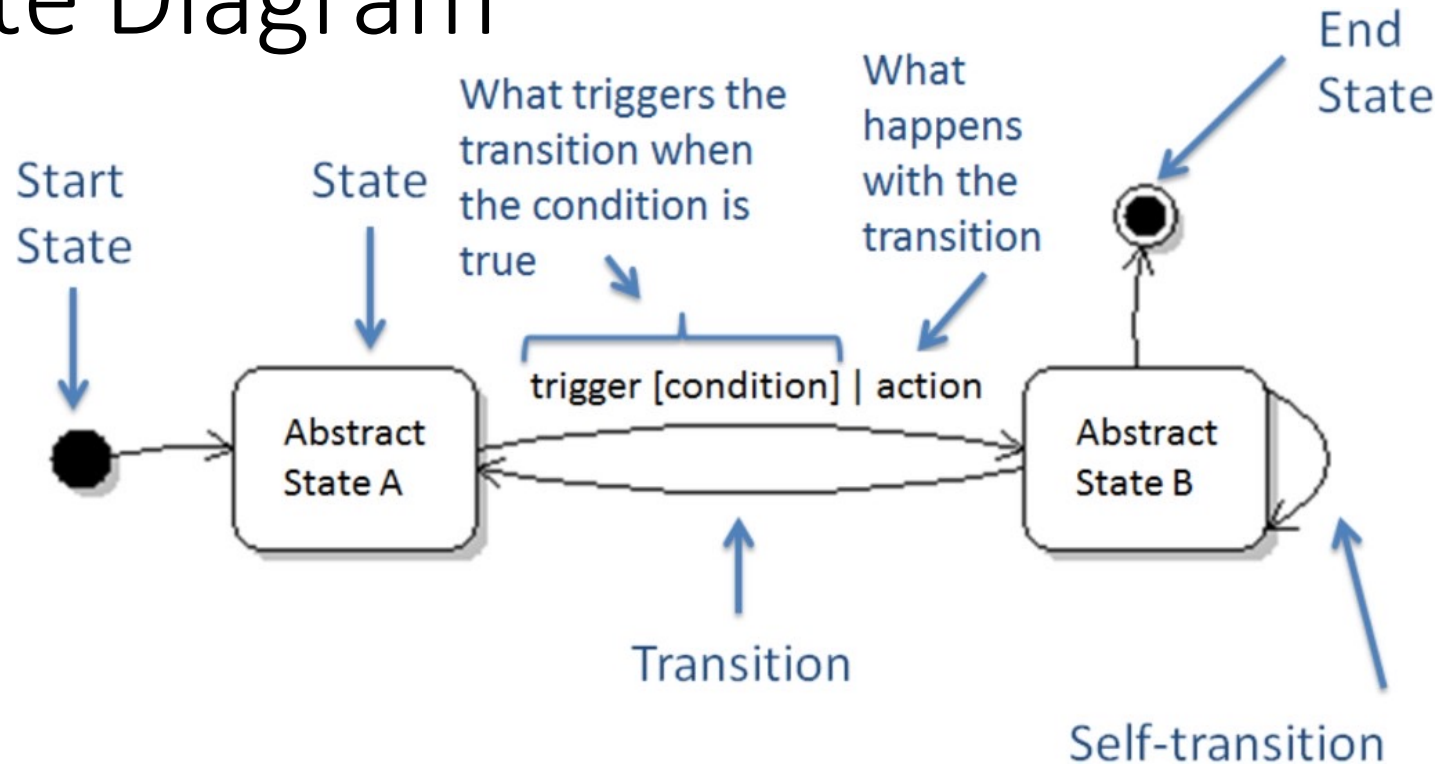
Class Diagram





Note: Ignore what the classes are for now and concentrate on what the relationships between the classes are and how they can be translated into code.

State Diagram



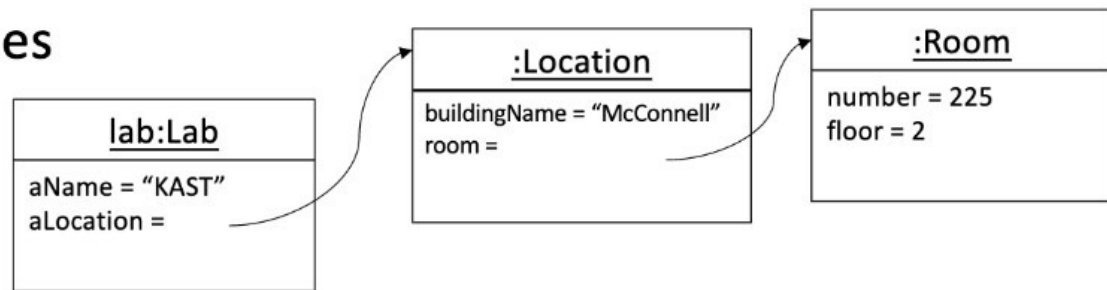
- Triggers are **potentially state-changing** methods.
- Every abstract state is the state of an object at a point in time.
- The state is always absolute, do not make it relative. It can not be “more” or “less” than something.
- There’s not necessarily an end state, but there is always a start state (initialization). For example, a list which has add and remove methods has no “end state”.
- Exceptions are not states.
- Add ALL possible legal transitions in the diagram.

State Diagram

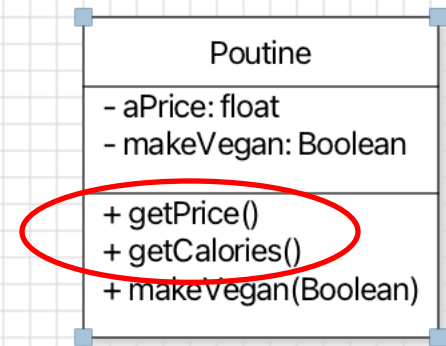
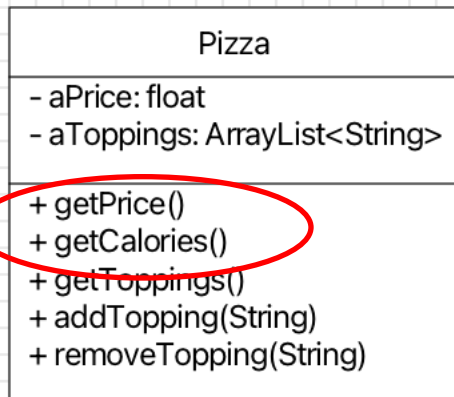
- Suppose you own a Warehouse.
- You can have a maximum of 100 products in your warehouse at any given point in time.
- You must have a minimum of 10 products in your warehouse at any given point in time.
- Customers can come and ask you for the price of products in your warehouse.
- Customers can also purchase items from your warehouse, as long as there are more than 10 items remaining.
- You can also restock your warehouse (i.e. add items) as long as it doesn't exceed 100.

Object Diagram

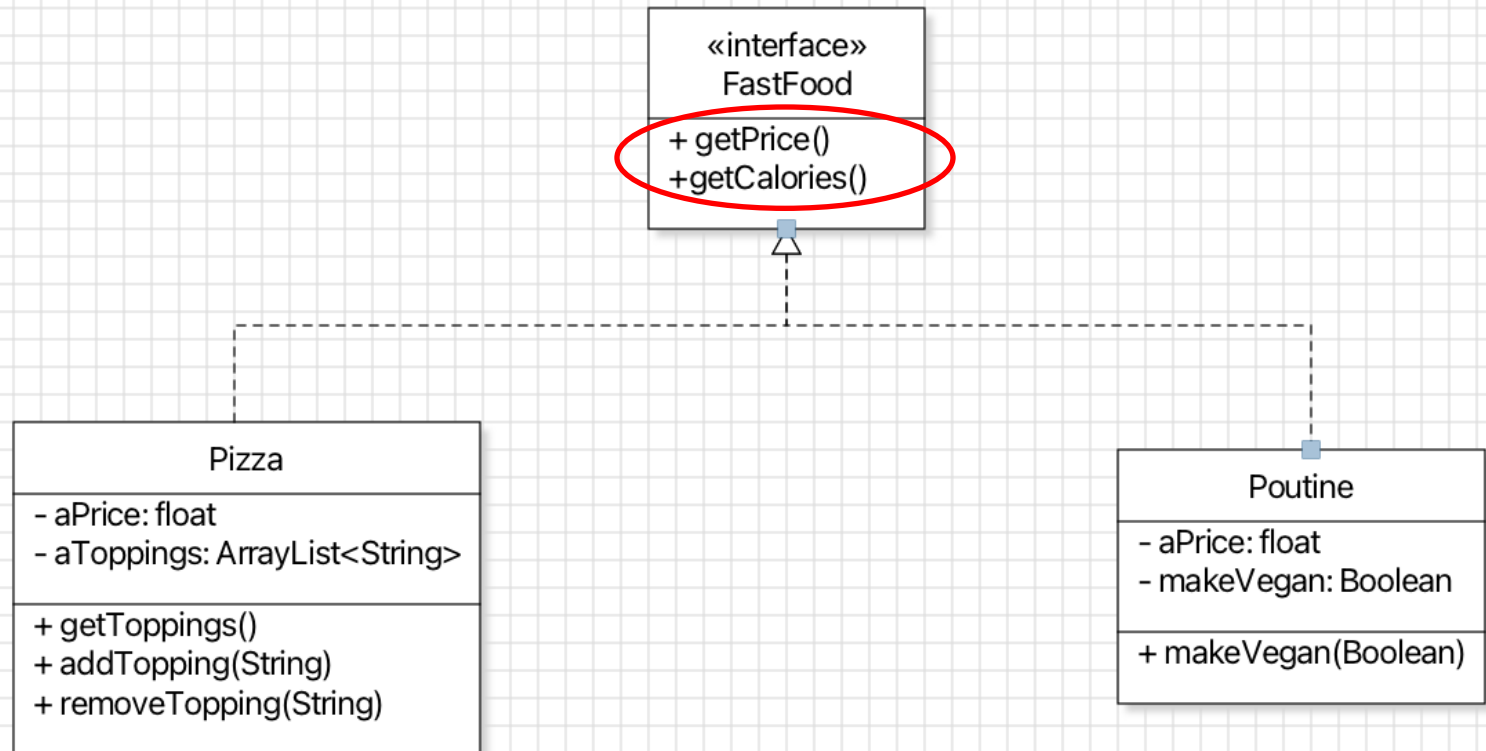
- Model the structure of the system **at a specific time**
- Complete or part of the system
- Include objects and data values



Interfaces



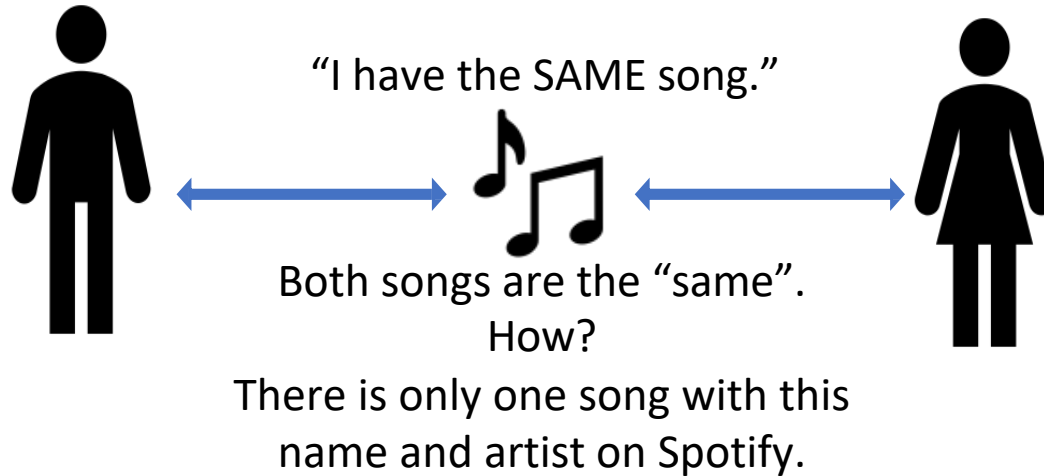
Interfaces



Flyweight vs Equality

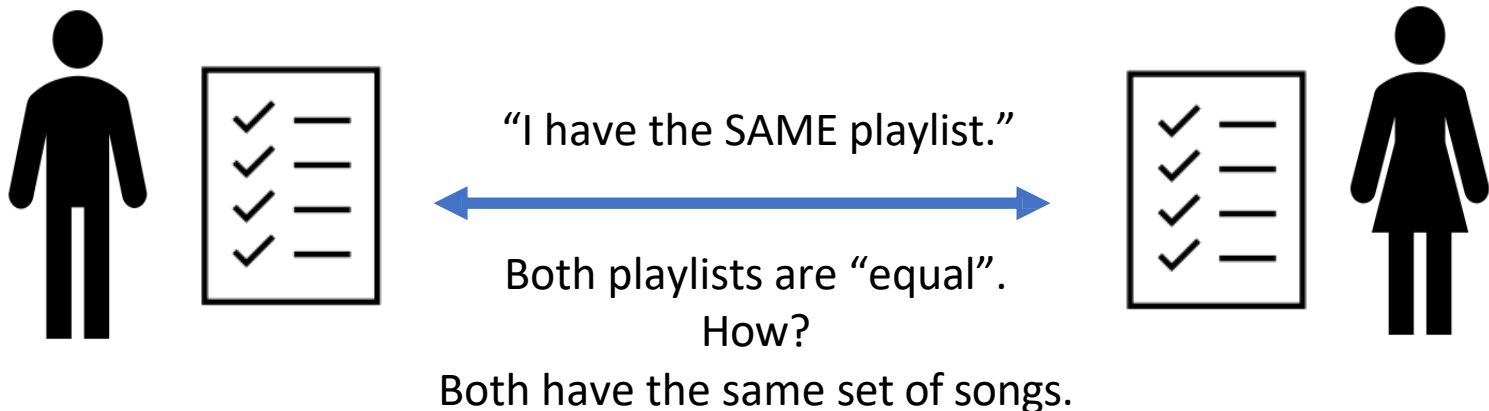
Flyweight:

- Implements object **uniqueness**



Equality:

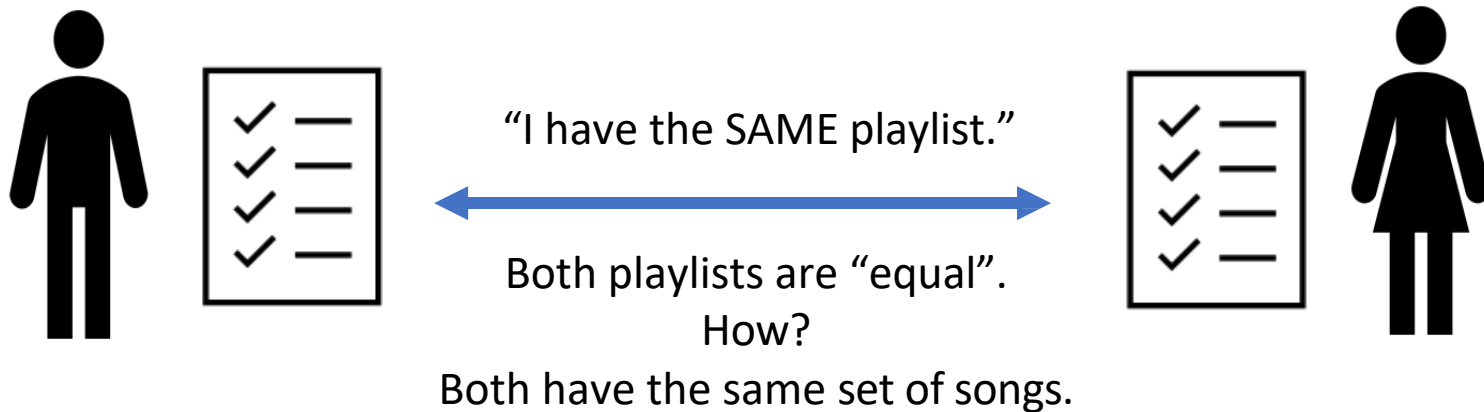
- Checks if two objects are **conceptually the same**



Overriding Equals and Hashcode

```
Playlist p1 = new Playlist("Alltime Classics");  
p1.addSong("Circle of Life");  
Playlist p2 = new Playlist("Alltime Favorites");  
p2.addSong("Circle of Life");
```

```
System.out.println(p1 == p2); False  
System.out.println(p1.equals(p2)); False
```



Overriding Equals

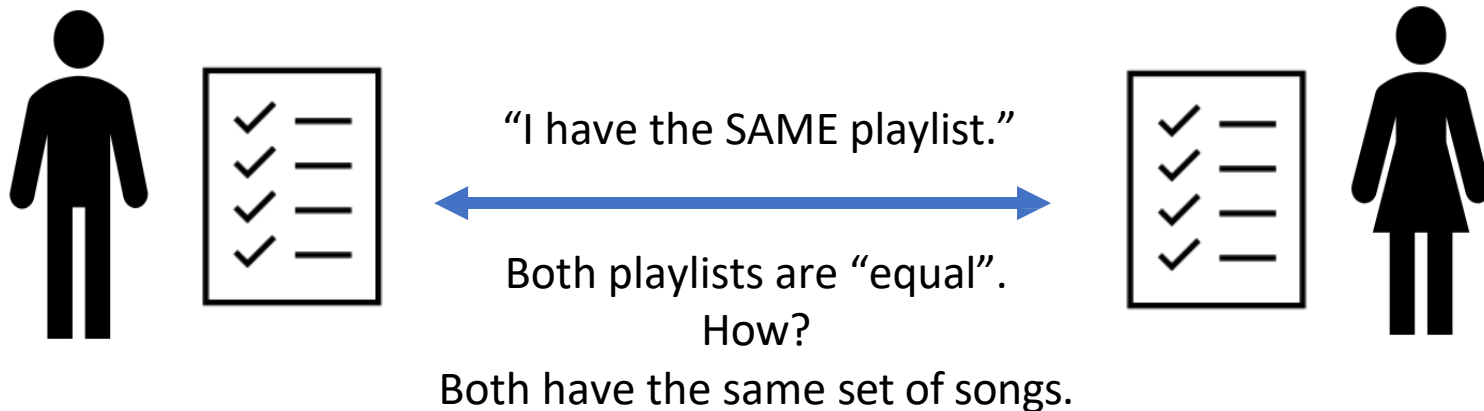
```
@Override
public boolean equals(Object o) {
    if (this == o)
        return true;
    if (o == null || getClass() != o.getClass())
        return false;
    PlayList playList = (PlayList) o;
    if (aList == null){
        if (playList.aList == null)
            return false;
    }
    return this.aList.equals(playList.aList);
}
```

The last line just checks if the lists have the same reference, instead we should check if the songs within the lists are also equal (i.e. have either the same reference, or have the same name and artist)

Overriding Equals and Hashcode

```
Playlist p1 = new Playlist("Alltime Classics");  
p1.addSong("Circle of Life");  
Playlist p2 = new Playlist("Alltime Favorites");  
p2.addSong("Circle of Life");
```

```
System.out.println(p1 == p2); False  
System.out.println(p1.equals(p2)); True
```



Overriding Hashcode

hashCode() is an integer value that represents the instance of a class.

Two objects which are “equal” must have the same hashcode.

But two objects which are non-equal may still have the same hashcode.

Some logic to calculate the hashcode:

```
/**
 * Equal playlists have the same hashcode
 */
@Override
public int hashCode() {
    return Objects.hash(aList);
}
```

Unit Testing

Still a regular class:

```
1  package testpackage;
2
3  + import static org.junit.Assert.*;
6
7  public class TestCase {
8
9  -   @Test
10     public void test() {
11         assertTrue(true);
12     }
13
14 }
```

(line 9) This is new ->

No main method?

Flash Light Example

Let's model a (simple) Flash Light 

It has one button that toggles it on and off:

```
1  package custompackage;
2
3  public class FlashLight {
4
5      private boolean isOn = false;
6
7      public void pressButton() {
8          this.isOn = ! this.isOn;
9      }
10
11     public boolean isOn() {
12         return this.isOn;
13     }
14 }
```

A Typical Unit Test

```
1 package custompackage;
2
3 import static org.junit.Assert.*;
4 import org.junit.Test;
5 import custompackage.FlashLight;
6
7 public class FlashLightTest {
8     @Test
9     public void testOffByDefault() {
10         FlashLight kitchenFlashLight = new FlashLight();
11         assertFalse(kitchenFlashLight.isOn());
12     }
13
14     @Test
15     public void testTurnsOnAfterOnePress() {
16         FlashLight kitchenFlashLight = new FlashLight();
17         kitchenFlashLight.pressButton();
18         assertTrue(kitchenFlashLight.isOn());
19     }
20
21     @Test
22     public void testTurnsBackOffAfterTwoPresses() {
23         FlashLight kitchenFlashLight = new FlashLight();
24         kitchenFlashLight.pressButton();
25         kitchenFlashLight.pressButton();
26         assertEquals(false, kitchenFlashLight.isOn());
27     }
28 }
29
```

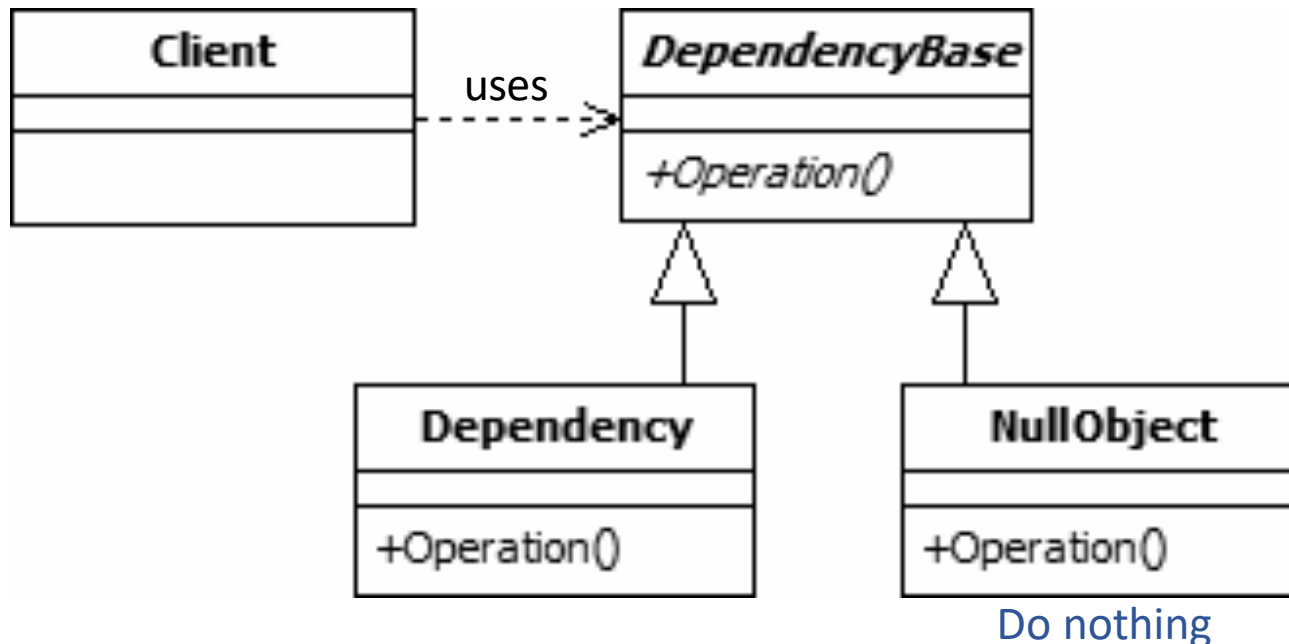
Unit Testing

A few things to remember:

- *A thorough* test suite is not just 100% coverage
- We do not test design by contract statements
- We do not test statements that print to console
- We don't modify a class just for testing

Null Object Pattern

- This pattern describes the “do-nothing” implementation.
- Useful to avoid client checking for null values before execution.
- Also when the program flow should not be interrupted if some values are undefined.



Null Object Pattern

```
public interface Router {  
    void route(Message msg);  
}
```

```
public class SmsRouter implements Router {  
    @Override  
    public void route(Message msg) {  
        // implementation details  
    }  
}
```

```
public class JmsRouter implements Router {  
    @Override  
    public void route(Message msg) {  
        // implementation details  
    }  
}
```

```
public class NullRouter implements Router {  
    @Override  
    public void route(Message msg) {  
        // do nothing  
    }  
}
```

Null Object Pattern

What happens if the message is null?

```
public class RoutingHandler {  
    public void handle(Iterable<Message> messages) {  
        for (Message msg : messages) {  
            Router router = RouterFactory.getRouterForMessage(msg);  
            router.route(msg);  
        }  
    }  
}
```

Method to determine which routing mechanism to use, based on the message.

- The client could have checked if `msg` is null and skipped routing it if it is null.
- But to relieve the client of this task, the `getRouterForMessage` will just return the `NullRouter`.
- So `router.route(msg)` will do nothing instead of throwing an error.

Null Object Pattern

What happens if the message is null?

```
public class RoutingHandler {  
    public void handle(Iterable<Message> messages) {  
        for (Message msg : messages) {  
            Router router = RouterFactory.getRouterForMessage(msg);  
            router.route(msg);  
        }  
    }  
}
```

Method to determine which routing mechanism to use, based on the message.

!

Careful!

If `msg` should never be null,
the Null Object Pattern would hide this bug.

Iterator

Iterable

- Both are interfaces

- Stores the iterator state – whether there is another element to be traversed, and what it is.
- An implementing class must override the `hasNext()` and `next()` methods
- Not assigned to an object/collection of objects.

- Represents a collection that can be traversed.
- An implementing class must override the `iterator()` method, which returns an `Iterator` object using which the collection is traversed.
- A collection is said to be `Iterable` if it can be traversed using the `Iterator`.

Comparable

Comparator

- Both are interfaces

- Useful when only one type of comparison is to be made
- An implementing class must override the `compareTo()` method.

- Useful when different types of comparison is to be made and the client can choose how to compare.
- An implementing class must override the `compare()` method.
- Example of Strategy Design Pattern

When would we use which design patterns?

- Singleton
- Flyweight
- Strategy Design Pattern

When would we use which design patterns?

Answers from the Activity:

- A Singleton Audio Manager to manage songs, playlists, etc. (Like iTunes)
- A Flyweight set of songs (or media types) so that everyone has access to the same audio files. (Useful in a shared system – say where iTunes is synced across a mobile and laptop)
- Strategy Design Pattern to filter songs based on their name (1st strategy), artist (2nd strategy), style (3rd strategy). (Useful to automatically create a playlist, e.g., songs by Adele)